Concourse: Four Design Questions

Background and questions for the next design conversation

Concourse research notes

July 17, 2026

Contents

1 What this document is

Concourse now simulates every queueing model we set out to support, and it computes parameter derivatives for most of them. Three capabilities remain open, and behind them sits one deeper question about how the system records its history. This document explains the background needed to think about those questions, then states each question in plain terms.

Each section builds on the ones before it. The background sections describe how the simulator works, what it writes down, and how derivative estimation uses what it wrote down. The question sections then say what is missing, why it is missing, and what a decision would need to settle.

2 Background

2.1 How the simulator runs a queueing model

A queueing model describes jobs that arrive, wait, get served, and leave. The simulator does not think in terms of jobs directly. It thinks in terms of *clocks*.

A clock is a pending event. When a job is being served, there is a clock for the moment its service will finish. When a source is active, there is a clock for the next arrival. Each clock has two pieces of information: a probability distribution for how long it will run, and the time it was started. We call the start time the enabling time.

The simulation is a race. All the active clocks run at once. The one that fires first wins. Firing a clock changes the state: a job moves, a queue grows or shrinks, a job leaves the system. The change can also start new clocks and cancel old ones. Then the race resumes among the clocks that remain. This loop is the entire simulation.

One rule governs everything the model tells the sampler. The model's whole statement about a clock is the pair (distribution, enabling time). The model never tells the sampler *how* to draw random numbers. It only declares what the law of the firing time is, given the history so far. The sampler is free to implement that law any way it likes.

2.2 The parameter vector

Models have tunable numbers: an arrival rate, a service rate, a patience rate. We collect them in a vector called θ. The value of θ enters the simulation in exactly one place: when the model builds a clock's distribution. Nothing else in the system reads θ. This discipline is what makes derivative estimation possible later, because there is only one door through which θ can affect anything.

2.3 The record

Each run writes down a record. The record is a list with one entry per firing. An entry holds three things: which clock fired, the time it fired, and any extra random values that the firing consumed (more on those below).

The record is the primary output of a run. Every statistic we report is computed by replaying the record: start from the initial state, apply the firings one at a time, and watch the state evolve. Replaying is cheap and it is exact, because the state change caused by a firing is a deterministic function once the recorded random values are given.

2.4 Auxiliary draws

Most randomness lives in the clocks. Two kinds do not.

First, *marks*. When a job arrives it may get random attributes: a size, a class. These are drawn at the moment of arrival, not raced as clocks.

Second, *random routing*. Some models flip a weighted coin to decide where a finished job goes next.

We call these auxiliary draws. The rule for them is strict: every auxiliary draw is taken from a named random stream and written into the record. Replay never draws again; it reads the recorded values back. This is what keeps replay exact even for models with marks and coin flips.

2.5 Re-declared clocks and segments

Usually a clock keeps one distribution for its whole life. Processor sharing breaks this pattern, and it is worth understanding why, because two of the open questions trace back to it.

Under processor sharing, all the jobs at a station are served at the same time, each at a fraction of the server's speed. When a new job arrives, the fraction shrinks for everyone. When a job finishes, the fraction grows. So the law of each remaining service time changes every time the number of jobs changes, even though the clocks themselves stay active.

We handle this by letting the model *re-declare* a clock. The clock keeps its original enabling time. What changes is the declared distribution, which now says: given how much work this job has already received, and given the current sharing fraction, here is the law of the remaining time. Each re-declaration starts a new *segment* in the clock's history. A clock that was never re-declared has one segment. A processor-sharing clock can have many.

Segments matter because the derivative machinery replays clock histories, and a multi-segment history is genuinely more complicated than a single-segment one.

2.6 Why we want derivatives

Suppose the average waiting time is $W(\theta)$. We often want $\partial W/\partial\theta$: how much the waiting time would change if we made service ten percent faster. Derivatives drive optimization, sensitivity analysis, and calibration against data.

The difficulty is that W is estimated by simulation. Running the model twice at two nearby values of θ and subtracting does not work well: the noise of the two runs swamps the small difference between them, unless the runs share their randomness in a carefully controlled way. The field has developed several estimators that compute the derivative from simulation output directly. Concourse supports four, through the ClockGradients package. They differ in what they need and in what they can handle.

2.7 The four estimators, in one paragraph each

Score function. Replay the record and compute how much more or less likely the same trajectory would have been at a slightly different θ. Multiply that sensitivity by the observed outcome and average over runs. This works for almost any model and any statistic. Its weakness is noise: the estimates have high variance.

Pathwise, also called IPA. Keep the order of firings fixed, and ask how each firing *time* would shift if θ moved a little. Slide the times, recompute the statistic, and read off the slope. This has low noise. Its weakness is blindness: if changing θ would change which clock wins a race,

IPA does not see that effect, and for statistics that jump when the order changes, IPA is simply wrong.

Branching. Run the simulation live. At a firing, make a copy of the whole running world and force a *different* clock to win in the copy. Compare the outcome of the copy with the outcome of the original. This directly measures the effect of order changes, which is exactly what IPA misses. Its weakness is cost: every branch point spawns clones.

Smoothed perturbation analysis, SPA. A compromise. Take the IPA estimate, then add a correction for the near-ties: the moments where two clocks almost fired at the same time, so a small change in θ could swap their order. The correction weighs each near-tie by how likely the swap was, using the hazard of the runner-up clock, and by how much the statistic would jump if the swap happened. It needs fewer clones than branching.

2.8 Cloning a world

Branching and SPA need to copy a running simulation. The copy must be exact: the same state, the same clocks, the same ages, and the same pending randomness. If you let the original and the copy both run forward with no other action, they must produce identical futures, firing for firing. We call this a coupled clone.

Sometimes we want the opposite. After cloning, we give the copy fresh randomness derived from a chosen seed. Then the copy's future is a fresh draw, independent of the original. And if two copies are given the *same* seed, their futures match each other while differing from the original. This controlled sharing of randomness is what makes the difference between two clones quiet: the shared randomness cancels, and only the effect we care about remains.

Concourse implements all of this today, with one restriction that leads directly to the first question.

3 Question 1: cloning worlds that use auxiliary draws

3.1 Where we are

The branchable world currently refuses any model that uses marks or random routing. The refusal is deliberate. Replay of a record is safe for these models, because the recorded values are read back. But a clone runs *forward*, past the end of any record. Its future arrivals need new sizes. Its future routing needs new coin flips. There is nothing recorded to read back, so the world would need live random streams for auxiliary draws, and we have not yet decided how those streams should behave under cloning.

3.2 What has to be decided

The clock randomness already has good cloning behavior, because the sampler keys each clock's stream by the clock's name. The auxiliary streams need the same three properties, and each one is a design choice.

First, *copying.* When a world is cloned, the auxiliary streams must be copied with their current positions, so that the clone's next size draw equals the original's next size draw. Otherwise the coupled clone would drift at the first arrival.

Second, *re-seeding.* When a clone is given fresh randomness, the auxiliary streams must be re-seeded too. If we re-seed only the clocks, the clone would replay the original's future sizes, and

the two would stay partly correlated when they should be independent.

Third, and hardest, *alignment*. Two same-seed clones are supposed to see matching randomness. But after a forced firing, the two clones can disagree about the order of later events. Suppose clone A sees its next two arrivals in the order job then job, while clone B, whose forced firing delayed something, sees an extra service completion in between. If size draws are handed out in the order they are requested, the two clones may consume the stream at different points, and their jobs stop matching. The design choice is what the draws should be keyed by. Keying by request order is simple and wrong. Keying by something stable, such as the arrival's position in that source's own sequence, keeps the fifth arrival of clone A matched with the fifth arrival of clone B, no matter what else happened in between.

There is also a smaller, separable question. If a mark's distribution depends on θ, the derivative of the mark itself becomes part of the answer, and the estimators need an extra term for it. Everything is simpler when marks do not depend on θ. We can choose to support θ-free marks first, and treat θ-dependent marks as a later step.

3.3 What a decision looks like

A decision here settles: where the auxiliary streams live (in the world object, alongside the sampler), what re-seed covers, and what the stream key is for each kind of draw. With those three fixed, the implementation is mechanical, and the certification harness can be extended to test the new obligations: clone-copies-marks, reseed-covers-marks, and same-seed clones agree on the sizes of corresponding jobs.

4 Question 2: SPA's correction for re-declared clocks

4.1 Where we are

SPA works today for models whose clocks keep one distribution for life. It refuses, with a clear message, any record that contains a multi-segment clock. So SPA runs on our FCFS and priority models, but not on processor sharing. The refusal is not an implementation gap. The mathematics of the correction term has not been worked out for the multi-segment case.

4.2 The problem in plain terms

Recall what the correction does. At a near-tie, two clocks could have fired in either order. The correction estimates two things: the probability rate at which the order would swap as θ moves, and the change in the statistic if it did swap.

The swap rate is built from the runner-up clock's hazard rate at the decision time. The hazard rate answers: given that this clock has survived until now, how likely is it to fire in the next instant. For a single-segment clock, the hazard comes straight from the one declared distribution and the clock's age.

For a re-declared clock, the current law is the latest segment's law, and the age must be measured against the right anchor. That part is not deep; we know the current declared law at every moment. The deeper issue is the paired-clone construction around the swap. The two clones continue the world after firing the pair in both orders, and the correctness argument for the correction assumes the surviving clock's remaining life is conditioned in a specific way. When the surviving clock can be re-declared by the very firing we forced — and under processor sharing

it always is, because forcing a service completion changes the sharing fraction for everyone — the existing argument no longer covers the construction. The weight, the conditioning, and possibly the variance of the estimator all need to be re-derived.

4.3 What has to be decided

Two things, one mathematical and one structural.

The mathematical piece: derive the boundary weight when the runner-up has a multi-segment history, and check whether the paired-clone construction still cancels the shared prefix, or needs modification. This is research, and a small hand-worked example (two jobs in a processor-sharing station) would be the right first step. It could confirm the natural guess, or show that a new term appears.

The structural piece: the refusal currently happens for the whole record. A record could contain many single-segment candidates and only a few re-declared ones. Once the mathematics is settled we should decide whether validity is a property of the whole record or of each candidate separately, and place the check where the decision says it belongs.

5 Question 3: IPA over processor-sharing records

5.1 Where we are

The IPA machinery already supports multi-segment records. The replay formulas that slide firing times around were written with segments in mind, and the score estimator, which shares the same record structure, passes its tests on processor sharing. What we have not done is test IPA itself on a processor-sharing record. So this question may be small. It is listed because "untested" and "working" are different claims, and because a failure would be informative.

5.2 What could go wrong

Two distinct risks.

The mechanical risk: sliding a firing time in a multi-segment clock means inverting the chain of segment laws. The inversion formulas exist and are exercised by other models, but our processor-sharing law is a custom distribution, and the inversion uses functions of it (the inverse survival function in particular) that should be checked under differentiation.

The statistical risk: IPA is only valid when the statistic responds smoothly to small shifts in the firing times. Time-integral statistics, such as the average number in system, are smooth in this sense. But processor sharing creates a subtlety: shifting one completion time changes the sharing fraction over an interval, which changes *other* completion times through the re-declarations. The frozen-order replay does propagate these effects through the segment chain, but whether the propagation is complete for our construction is exactly what a test against finite differences would reveal.

5.3 What has to be decided

Very little needs deciding in advance. The right move is an experiment: run IPA on the processor-sharing model against finite differences, the same comparison every other estimator passed. If it matches, we write down the validity claim (which statistics, which couplings) and add the test to the charter. If it does not match, the failure will point at either the mechanical risk or the

statistical risk, and that outcome feeds Question 2, because both trace to the same underlying object: derivative flow through re-declaration boundaries.

6 Question 4: should the record store segment boundaries?

6.1 Where we are

This question sits underneath the previous two.

Today the record stores only firings: clock, time, auxiliary values. It does not store when clocks were enabled, and it does not store when clocks were re-declared. All of that is *reconstructed*: a bookkeeping walk re-runs the model's own rules over the firing list and rebuilds each clock's history, including its segments. Re-declarations are detected by comparing distributions: after each firing, the walk rebuilds each active clock's declared law and checks whether it changed.

6.2 The case for reconstruction

Reconstruction keeps the record minimal, and it forces the model's rules to be the single source of truth. There is also a real audit benefit: when the sampler's own notion of the clock history is compared against the reconstruction and they agree, we have checked, on every run, that the model rules and the simulation actually match. Storing the history instead would remove that cross-check, because the record would just repeat what the simulator believed.

6.3 The case for storing

Reconstruction has costs. It walks the whole record and rebuilds every active clock's distribution after every firing, which is the most expensive part of preparing a record for the estimators. It also rests on an assumption: that a re-declaration always produces a visibly different distribution. If a re-declaration ever produced an equal distribution — same law, arrived at for a different reason — the walk would not see it. Today that case cannot arise, because our only re-declaration source is a change in the sharing fraction, which always changes the law. But the assumption is load-bearing, and it is the kind of assumption that a future feature could silently break.

Storing segment boundaries in the record would make preparation cheap and the assumption unnecessary. The record would grow, and the audit would weaken.

6.4 What has to be decided

This is a choice about where truth lives. Either the model's rules are authoritative and the record is minimal evidence, or the record is authoritative and the rules are one producer of it. The right answer may depend on the outcomes of Questions 2 and 3: if re-declaration boundaries become central to more estimators, the argument for recording them directly gets stronger. The recommendation is to hold this question open until at least the Question 3 experiment is done, and then decide with that data in hand.

7 Summary

Question	Kind	First step	What it unlocks
1. Auxiliary draws in clones	engineering	pick the stream keys	branching for SITA-style models
2. SPA with re-declaration	research	two-job worked example	SPA for processor sharing
3. IPA on PS records	experiment	finite-difference test	completes the estimator table
4. Segments in the record	architecture	wait for 2 and 3	simpler, faster estimation prep

A reasonable order: run the Question 3 experiment first, because it is cheap and its outcome informs Questions 2 and 4. Then settle Question 1, which is self-contained engineering. Then take up Question 2 with whatever Question 3 revealed. Then revisit Question 4.